Day 30

Components + JSX

What is a Component in React?

In React, a component is an independent reusable piece of UI.

Instead of writing one huge page in a single file, React allows us to split the page into smaller reusable parts.

Example:

A website page can be divided into components like:

- Header
- Navbar
- Sidebar
- Product Card
- Footer

Each part can be created separately and reused.

Simple Meaning

Component = Small reusable section of a webpage

Why Components are Important?

Components help:

reduce repeated code
make project clean
improve readability
reuse same UI many times
easier maintenance

Real Example

In your LMS or job portal project:

A course card can be reused for all courses.

A job card can be reused for all jobs.

A navbar can appear on every page.

Types of Components

Main beginner type:

Functional Component

This is the most commonly used component.

```
function Welcome() {
   return <h1>Hello React</h1>;
}
```

What is JSX?

JSX means JavaScript XML.
It allows writing HTML-like code inside JavaScript.

Example

```
const element = <h1>Welcome</h1>;
```

This looks like HTML, but it is inside JavaScript file.

Why JSX?

Without JSX:

```
React.createElement("h1", null, "Hello");
```

With JSX:

```
<h1>Hello</h1>
```

JSX is easier to read and write.

5. Rules of JSX

Important rules:

Rule 1

Must return only one parent element

Correct:

```
return (
  <div>
    <h1>Hello</h1>
    <p>Welcome</p>
  </div>
);
```

Wrong:

```
return (
  <h1>Hello</h1>
  <p>Welcome</p>
);
```

Rule 2

Use className instead of class

```
<div className="box"></div>
```

Rule 3

All tags must be closed

```
<img src="image.jpg" />
```

Example Program

```
function App() {
  return (
    <div>
```

```
        <h1>My First React App</h1>
        <p>Learning Components and JSX</p>
      </div>
  );
}
export default App;
```

Creating Multiple Components

```
function Header() {
  return <h1>Header Section</h1>;
}

function Footer() {
  return <p>Footer Section</p>;
}
function App() {
  return (
    <div>
      <Header />
      <Footer />
    </div>
  );
}
```

Tasks (Detailed)

Task 1: Create a Simple Heading Component

Create a React component named Heading that displays a heading text like "Welcome to React Learning".

Task 2: Create a Paragraph Component

Create a component that shows a paragraph explaining your learning journey.

Task 3: Create a Button Component

Create a reusable button component with any label such as "Click Here".

Task 4: Combine Multiple Components

Create 3 components:

- Header
- Content
- Footer

Display all inside App.js.

Task 5: Create Student Information Component

Create a component that displays:

- student name
- age
- course

Task 6: Create Product Card Component

Create a product card with:

- product name
- price
- category

Use JSX structure properly.

Task 7: Create Employee Card Component

Create employee details card:

- employee name
- role
- department

Task 8: Create Course Card Layout

Create a course component that shows:

- course title
- trainer name
- duration

Task 9: Create Navbar Component

Create navigation menu component with:

- Home
- About
- Contact

Task 10: Build Mini Profile Page

Create a small profile page using multiple components:

- Header
- Profile Card
- Skills
- Footer

Mini Projects (Detailed)

Mini Project 1: Student Profile App

Create a React application for student details.

Features:

- header section
- student card

- course details
- footer

Use separate components for each section.

Mini Project 2: Product Display App

Build a product display page.

Include:

- title
- multiple product cards
- price
- category
- footer

Use reusable product component.

Mini Project 3: Employee Directory App

Create employee management frontend.

Display:

- employee name
- department
- role
- email

Use JSX and components.

Mini Project 4: Course Listing App

Build a course listing page.

Display:

- course title
- trainer
- duration
- fees

Use multiple reusable components.

Day 31

Props

What are Props in React?

In React, **props** means **properties**.

Props are used to **pass data from one component to another component**.

Usually:

- Parent component sends data
- Child component receives data

Simple Meaning

Props = Data passed between components

Why Props are Important?

Without props:

- components show fixed content only

With props:

- same component can display different data dynamically

This makes components reusable.

Real Example

Suppose you create one StudentCard component.

Without props:

- only one student shown

With props:

- same component can show many students

Example:

- Arun
- Kumar
- Priya
- Divya

Using same component.

Basic Props Syntax

Parent component:

```
<Student name="Arun" age="21" course="React" />
```

Child component:

```
function Student(props) {
   return (
      <div>
         <h2>{props.name}</h2>
         <p>{props.age}</p>
         <p>{props.course}</p>
      </div>
   );
}
```

How Props Work?

Flow:

Parent → sends data

Child → receives data

Example

```
function App() {
   return (
      <Student name="Kumar" />
   );
}
```

App passes data.

Student receives data.

Accessing Props

There are two common ways.

Method 1: props object

```
function User(props) {
   return <h1>{props.name}</h1>;
}
```

Method 2: Destructuring

```
function User({ name }) {
   return <h1>{name}</h1>;
}
```

This is cleaner.

Multiple Props

You can send many values.

```
<Product title="Laptop" price="50000" category="Electronics" />
```

Receive

```
function Product(props) {
   return (
      <div>
         <h2>{props.title}</h2>
         <p>{props.price}</p>
         <p>{props.category}</p>
      </div>
   );
}
```

Reusable Component Example

```
function Employee(props) {
   return (
      <div>
         <h3>{props.name}</h3>
```

```
      <p>{props.role}</p>
    </div>
  );
}
function App() {
  return (
    <div>
      <Employee name="Ravi" role="Developer" />
      <Employee name="Meena" role="Designer" />
    </div>
  );
}
```

Why Reusable?

Same component reused with different data.

This reduces repeated code.

Full Example

```
function Course(props) {
  return (
    <div>
      <h2>{props.title}</h2>
      <p>{props.trainer}</p>
      <p>{props.duration}</p>
    </div>
  );
}

function App() {
  return (
    <div>
      <Course title="React" trainer="Alagu" duration="30 Days" />
      <Course title="Django" trainer="John" duration="45 Days" />
    </div>
  );
}
```

Real Project Usage

Props are heavily used in:

- eCommerce product cards

- LMS course cards
- job listings
- employee management
- dashboards
- blogs

Since you've built LMS and job portal projects, props are exactly what makes those card components reusable in React.

Tasks (Detailed)

Task 1: Create Student Component with Props

Create a component that receives:

- name
- age
- course

Task 2: Create Employee Component

Pass:

- employee name
- role
- department

Use props.

Task 3: Create Product Component

Pass:

- product name
- price
- category

Task 4: Create Course Component

Pass:

- course title
- trainer
- duration

Task 5: Create User Profile

Pass:

- username
- city
- profession

Task 6: Create Multiple Student Cards

Use same component to display 3 students.

Task 7: Create Multiple Product Cards

Display 5 products using one reusable component.

Task 8: Use Destructuring

Rewrite props using destructuring syntax.

Task 9: Create Job Card Component

Pass:

- job title
- company
- salary

Task 10: Build Full Dashboard Cards

Create:

- employee cards
- product cards
- course cards

Using props.

Mini Projects (Detailed)

Mini Project 1: Student Management UI

Build student listing page.

Each student card should show:

- name
- course
- age
- city

Use props.

Mini Project 2: Product Listing UI

Build product page.

Each product card should show:

- image
- name
- price
- category
- Use reusable props.

Mini Project 3: Job Portal Cards

Build job portal interface.

Each card should show:

- company
- role
- salary

- location

Use props.

Mini Project 4: LMS Course Cards

Build course listing page.

Each card should show:

- course title
- trainer
- duration
- fees

Use props.

Day 32: State

What is State in React?

In React, **state** is used to store data inside a component and update the UI whenever that data changes.

Props pass data from parent to child.

State stores data **inside the component itself**.

Simple Meaning

State = Component's own data storage

Why State is Important?

Without state:

- page shows static content only

With state:

- page can change dynamically

Examples:

- counter app
- form input
- dark/light mode
- cart item count
- login status
- todo app

Real Example

Suppose you create a counter.

Button click:

- number increases

This changing value is stored in state.

useState Hook

React uses useState() to manage state in functional components.

Basic syntax:

```
import { useState } from "react";
```

Example

```
const [count, setCount] = useState(0);
```

Understanding Syntax

```
const [value, setValue] = useState(initialValue);
```

Here:

Part	Meaning
value	current state
setValue	function to update state
initialValue	starting value

Example

```
const [name, setName] = useState("Alagu");
```

- current value = Alagu
- update function = setName

Counter Example

```
import { useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => setCount(count + 1)}>Increase</button>
    </div>
  );
}
export default App;
```

How It Works

Initial:

- count = 0

Button click:

- count + 1

UI updates automatically.

State with Text

```
import { useState } from "react";

function App() {
   const [message, setMessage] = useState("Hello");

   return (
      <div>
         <h1>{message}</h1>
         <button onClick={() => setMessage("Welcome")}>Change</button>
      </div>
   );
}
```

State with Input

```
import { useState } from "react";
function App() {
   const [name, setName] = useState("");
  return (
      <div>
         <input onChange={(e) => setName(e.target.value)} />
         <h2>{name}</h2>
      </div>
   );
}
```

Why State is Powerful?

Because it updates UI automatically.

No manual DOM manipulation needed.

Plain JS:

- document.getElementById()

React:

- state handles updates automatically

Props vs State

Props	State
passed from parent	stored inside component
read-only	changeable
external data	internal data

Full Example

```
import { useState } from "react";
function App() {
   const [student, setStudent] = useState("Arun");
return (
      <div>
         <h1>{student}</h1>
          <button onClick={() => setStudent("Kumar")}>
            Change Student
         </button>
      </div>
   );
}
```

Real Project Usage

State is used in:

- login forms
- cart systems
- todo apps
- search filter
- LMS dashboards
- job applications
- employee forms

For projects like your employee management system or job portal, state is what manages user input and live updates in React frontend.

Tasks (Detailed)

Task 1: Create Counter App

Create button:

- increase count

Display count using state.

Task 2: Create Text Changer

Show message.

Button click changes message.

Task 3: Create Name Display

Input field.

Type name.

Display below.

Task 4: Create Age Updater

Button click increases age.

Task 5: Toggle Status

Show:

- Online
- Offline

Using state.

Task 6: Student Profile State

Store:

- name
- course
- city

Update on button click.

Task 7: Product Price State

Store price.

Update when discount applied.

Task 8: Theme Toggle

Light/Dark mode state.

Task 9: Login Status

Show:

- Logged In
- Logged Out

Task 10: Full User Form

Store:

- name
- email
- phone

Using state.

Mini Projects (Detailed)

Mini Project 1: Counter Application

Features:

- increase
- decrease
- reset

Use state.

Mini Project 2: Live Profile Updater

User inputs:

- name
- age

- city

Display instantly.

Mini Project 3: Theme Switcher

Button toggles:

- light theme
- dark theme

Using state.

Mini Project 4: Student Registration UI

Input fields:

- student name
- course
- email

Display entered data live.

Day 33

Conditional Rendering

What is Conditional Rendering in React?

In React, conditional rendering means **showing different UI content based on a condition**.

It works similar to JavaScript conditions (if, else, ternary operator), but inside React components.

Simple Meaning

Conditional Rendering = Displaying UI based on condition

Why Conditional Rendering is Important?

In real applications, some content should appear only when certain conditions are true.

Examples:

- show login button if user not logged in
- show logout button if user logged in
- show “No products” if product list empty
- show admin panel only for admin
- show loading message while API data loads

Real Example

Suppose user is logged in.

Display:

Welcome User
Logout Button
If user is not logged in:
Login Button
This is conditional rendering.

Using if Statement

Basic example:

```
function App() {
   let isLoggedIn = true;

   if (isLoggedIn) {
      return <h1>Welcome User</h1>;
   } else {
      return <h1>Please Login</h1>;
   }
}
```

Using Ternary Operator

Most commonly used in React.

Syntax:

```
condition ? trueValue : falseValue
```

Example:

```
function App() {
   let isLoggedIn = false;

   return (
      <div>
         {isLoggedIn ? <h1>Welcome</h1> : <h1>Login First</h1>}
      </div>
   );
}
```

Using && Operator

Used when only one condition needed.

Example:

```
function App() {
   let showMessage = true;

   return (
```

```
      <div>
        {showMessage && <p>Message displayed</p>}
      </div>
    );
}
```

Meaning

If true → show

If false → hide

Conditional Rendering with State

Best real usage.

```
import { useState } from "react";

function App() {
  const [status, setStatus] = useState(false);

  return (
    <div>
      <button onClick={() => setStatus(!status)}>Toggle</button>

      {status ? <h1>Online</h1> : <h1>Offline</h1>}
    </div>
  );
}
```

How It Works

Initial:

- false → Offline

Button click:

- true → Online

Real Project Use Cases

Conditional rendering is used in:

- login systems
- dashboards
- employee portals
- course access
- job application pages
- eCommerce cart
- API loading screens

This is especially useful for your LMS and employee management projects (show dashboard only after login, show admin options conditionally).

Full Example

```
import { useState } from "react";

function App() {
  const [show, setShow] = useState(true);

  return (
    <div>
      <button onClick={() => setShow(!show)}>
        Toggle Content
      </button>

      {show ? <h2>Visible Content</h2> : <h2>Hidden Content</h2>}
    </div>
  );
}

export default App;
```

Tasks (Detailed)

Task 1: Login Message

Display:

- Welcome if logged in
- Please login if not

Task 2: Online Status

Display:

- Online
- Offline

Task 3: Age Eligibility

Show:

- Eligible
- Not Eligible

Task 4: Product Availability

Display:

- In Stock
- Out of Stock

Task 5: Course Access

Display:

- Access Granted
- Access Denied

Task 6: Theme Message

Show:

- Light Mode
- Dark Mode

Task 7: Student Result

Show:

- Pass
- Fail

Task 8: Button Toggle Content

Button click:

- show/hide paragraph

Task 9: API Loading Screen

Display:

- Loading...
- Data Loaded

Task 10: Dashboard Access

Show:

- Admin Dashboard
- User Dashboard

Based on role.

Mini Projects (Detailed)

Mini Project 1: Login UI

Create login system.

Display:

- login page
- dashboard

Based on state.

Mini Project 2: Theme Toggle App

Button click:

- light mode
- dark mode

Use conditional rendering.

Mini Project 3: Product Stock Checker

Display products.
Show:

- in stock
- out of stock

Mini Project 4: Student Result App
Input marks.
Display:

- pass
- fail

Conditionally.

Day 34

Lists + Events

What are Lists in React?
In React, lists are used to display multiple items dynamically from an array. Instead of writing repeated HTML manually, React can loop through data and render UI automatically.
Simple Meaning
List = Displaying multiple data items on screen
Why Lists are Important?
Used in almost all applications:

- product cards
- student lists
- course lists
- employee details
- job listings
- cart items
- notifications

Real Example
Suppose you have 5 courses:

- HTML
- CSS
- JavaScript
- React
- Django

Instead of manually writing 5 <li> tags, React can generate automatically.

Using map() for Lists

React uses JavaScript map() method.

Example:

```
function App() {
  const courses = ["HTML", "CSS", "JavaScript"];

  return (
    <ul>
      {courses.map((course, index) => (
        <li key={index}>{course}</li>
      ))}
    </ul>
  );
}
```

What is key?

Each list item needs a unique key.

Why?

React identifies items efficiently.

```
<li key={index}>{course}</li>
```

Lists with Objects

```
function App() {
  const students = [
    { name: "Arun", course: "React" },
    { name: "Kumar", course: "Django" }
  ];

  return (
    <div>
      {students.map((student, index) => (
        <div key={index}>
          <h2>{student.name}</h2>
          <p>{student.course}</p>
        </div>
      ))}
    </div>
  );
}
```

What are Events in React?

Events are user actions.

Examples:

- button click
- typing input
- submitting form
- mouse hover

React handles events using JSX syntax.

Simple Meaning

Event = User interaction

Common Events in React

Common events:

- onClick
- onChange
- onSubmit
- onMouseOver
- onKeyUp

onClick Event

```
function App() {
  function showMessage() {
    alert("Button clicked");
  }

  return (
    <button onClick={showMessage}>Click</button>
  );
}
```

onChange Event

Used for input fields.

```
import { useState } from "react";

function App() {
  const [name, setName] = useState("");

  return (
    <input
      onChange={(e) => setName(e.target.value)}
```

```
      />
    );
}
```

Combining Lists + Events

Example:

```
function App() {
  const products = ["Mobile", "Laptop", "TV"];

  function showProduct(item) {
    alert(item);
  }

  return (
    <ul>
      {products.map((product, index) => (
        <li key={index} onClick={() => showProduct(product)}>
          {product}
        </li>
      ))}
    </ul>
  );
}
```

Real Project Usage

Lists + events are used in:

- eCommerce products
- LMS course display
- employee list
- task manager
- shopping cart
- job portal

Since you already build product and job portal style apps, this is the core for displaying multiple cards and handling clicks.

Tasks (Detailed)

Task 1: Display Course List

Create array of 5 courses.

Display using map().

Task 2: Display Student List

Create array of students.
Show name list.

Task 3: Button Click Event

Create button.
Show alert on click.

Task 4: Input Event

Take input.
Display entered text.

Task 5: Product List

Display products using map.

Task 6: Click Product Name

Click product → show alert.

Task 7: Employee Cards

Display employee objects in cards.

Task 8: Delete Item Button

Create button inside list.

Task 9: Course Search

Input field + filter list.

Task 10: Dynamic Task Manager

Add task + display list + delete.

Mini Projects

Mini Project 1: Student List App

Display:

- name
- course
- city

Using map.

Mini Project 2: Product Viewer

Display:

- product name
- price
- category

Click product → show details.

Mini Project 3: Employee Directory

Display employee cards.
Add button:

- show employee details

Mini Project 4: Course Dashboard

Display all course cards.

Button:

- enroll course

Day 35

Mini Projects

Mini Project 1: Employee Management App

Features:

- add employee
- display list
- delete employee

Mini Project 2: Product Management App

Features:

- add product
- display products
- delete product

Mini Project 3: Course Enrollment App

Features:

- add course
- show list
- remove course

Mini Project 4: Task Manager App

Features:

- add task
- delete task
- count tasks

Day 37

API Calls in React

What is API Call in React?

In React, API call means requesting data from a server and displaying it inside a React component.React applications often connect to backend services such as:

- Django APIs
- external public APIs
- databases through backend
- authentication services

Simple Meaning

API Call = Getting data from server inside React app

Why API Calls are Important?

Most real-world applications use APIs:

- eCommerce product data
- LMS course data
- employee records
- job listings
- weather info
- food delivery menus
- user profiles

Without API:

- static app only

With API:

- dynamic real-time data

Example Real Flow

React frontend

⬇

API request

⬇

Server sends JSON

⬇

React displays data

Which Hook is Used?

API calls are usually done using:

useEffect()

Because API should load when component renders.

Also use:

useState()

To store fetched data.

Basic Syntax

```
import { useEffect, useState } from "react";
```

Basic API Call Example

```
import { useEffect, useState } from "react";
function App() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
```

```
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(response => response.json())
      .then(data => setUsers(data));
  }, []);
  return (
    <div>
      <h1>User List</h1>
    </div>
  );
}
export default App;
```

Understanding useEffect

```
useEffect(() => {
  // code
}, []);
```

[] means:

Run only once when page loads.

Perfect for API calls.

Displaying API Data

```
function App() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(res => res.json())
      .then(data => setUsers(data));
  }, []);

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

Loading Message

While API data loads:

```
const [loading, setLoading] = useState(true);
```

Example:

```
useEffect(() => {
   fetch("https://jsonplaceholder.typicode.com/users")
      .then(res => res.json())
      .then(data => {
         setUsers(data);
         setLoading(false);
      });
}, []);
```

Render

```
{
   loading ? <p>Loading...</p> : <p>Data Loaded</p>
}
```

Error Handling

```
fetch("https://wrongurl.com")
   .then(res => res.json())
   .catch(error => console.log(error));
```

Full Example

```
import { useEffect, useState } from "react";

function App() {
   const [products, setProducts] = useState([]);

   useEffect(() => {
      fetch("https://fakestoreapi.com/products")
         .then(res => res.json())
         .then(data => setProducts(data));
   }, []);

   return (
      <div>
         {products.map(product => (
               <div key={product.id}>
               <h2>{product.title}</h2>
               <p>{product.price}</p>
```

```
            </div>
          ))}
        </div>
    );
}
export default App;
```

Real Project Usage

API calls are used in:

- LMS course list
- employee management
- product catalog
- job portal
- dashboards
- invoice systems
- food delivery apps

Since you're building full-stack apps, this is exactly how React frontend connects to your Django backend APIs.

Tasks (Detailed)

Task 1

Fetch users API and print in console

Task 2

Display user names in React

Task 3

Display user emails

Task 4

Show first 5 users

Task 5

Show loading message

Task 6

Handle API error

Task 7

Display product API

Task 8

Create product cards

Task 9

Search API data

Task 10

Build API dashboard

Mini Projects (Detailed)

Mini Project 1: User Directory

Display:

- name
- email
- phone

Mini Project 2: Product App

Display:

- product title
- image
- price

Mini Project 3: Course Dashboard

Fetch course API.

Display:

- course name
- trainer
- duration

Mini Project 4: Employee List App

Display:

- employee name
- role
- department

Day 38

Routing

What is Routing in React?

In React, routing means navigating between different pages/components **without reloading the entire website**.

It is commonly handled using React Router.

Simple Meaning

Routing = Moving between pages inside a React app

Why Routing is Important?

In multi-page style apps, users need to move between sections like:

- Home
- About
- Contact

- Products
- Dashboard
- Login

Without routing:

- only one page

With routing:

- multiple pages inside single React app

Real Example

In your projects like LMS or job portal:

Pages can be:

- Home
- Courses
- Jobs
- Profile
- Dashboard

Routing helps switch between them.

Install React Router

Install package:

npm install react-router-dom

Basic Setup

Import router:

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
```

Simple Routing Example

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
function Home() {
  return <h1>Home Page</h1>;
}
function About() {
  return <h1>About Page</h1>;
}

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
```

```
            <Route path="/about" element={<About />} />
          </Routes>
        </BrowserRouter>
    );
}
export default App;
```

How It Works

/

Shows Home component

/about

Shows About component

Navigation Links

Use Link component.

```
import { Link } from "react-router-dom";
```

Example:

```
<Link to="/">Home</Link>
<Link to="/about">About</Link>
```

Full Example

```
function Navbar() {
    return (
        <nav>
            <Link to="/">Home</Link>
            <Link to="/about">About</Link>
        </nav>
    );
}
```

Multiple Pages

Example structure:

```
src/
|
├── App.js
├── Home.js
├── About.js
├── Contact.js
├── Navbar.js
```

Dynamic Route Example

<Route path="/student/:id" element={<Student />} />

Used for:

- product details
- course details
- job details
- employee profile

Real Project Usage

Routing is used in:

- eCommerce
- LMS
- job portal
- employee management
- admin panel
- dashboards
- blogs

This is essential for your full-stack apps because it lets React frontend behave like a multi-page app while talking to Django backend.

Tasks

Task 1

Install React Router

Task 2

Create Home page

Task 3

Create About page

Task 4

Create Contact page

Task 5

Add navigation links

Task 6

Create Product page

Task 7

Create Student page

Task 8

Create Dashboard page

Task 9

Create Login page

Task 10

Create full routing app

Mini Projects

Mini Project 1: Student App Routing

Pages:

- Home
- Students
- Add Student
- About

Mini Project 2: Product App Routing

Pages:

- Home
- Products
- Cart
- Contact

Mini Project 3: LMS Routing

Pages:

- Home
- Courses
- Trainers
- Dashboard

Mini Project 4: Employee System Routing

Pages:

- Home
- Employees
- Attendance
- Leave

Day 39

Context API

What is Context API?

In React, Context API is used to share data between many components **without passing props manually at every level**.

Normally, data is passed using props:

Parent → Child → Grandchild → Next child

When the component tree becomes large, this becomes difficult. Context API solves that.

Simple Meaning

Context API = Share data globally inside React app

Why Context API is Needed?

Suppose your app has:

- App
- Dashboard
- Sidebar
- Profile
- Settings

If username needs to reach Profile:

Without Context:

- pass props through every component

With Context:

- any component can access directly

Real Example

Common shared data:

- logged-in user
- theme (dark/light)
- cart items
- language
- authentication
- settings

Problem Without Context

Example:

App → Parent → Child → SubChild

Passing props through many levels is called:

Prop Drilling

Context API avoids prop drilling.

Creating Context

Basic syntax:

```
import { createContext } from "react";
```

Create context:

```
const UserContext = createContext();
```

Provider

Provider shares data.

```
<UserContext.Provider value="Alagu">
```

Example

```
import { createContext } from "react";
export const UserContext = createContext();
function App() {
  return (
    <UserContext.Provider value="Alagu">
      <Profile />
    </UserContext.Provider>
  );
}
```

useContext Hook

To receive data:

```
import { useContext } from "react";
```

Example

```
function Profile() {
  const user = useContext(UserContext);

  return <h1>{user}</h1>;
}
```

Full Example

```
import { createContext, useContext } from "react";

const ThemeContext = createContext();

function Child() {
  const theme = useContext(ThemeContext);
  return <h1>{theme}</h1>;
}
function App() {
  return (
    <ThemeContext.Provider value="Light Mode">
      <Child />
    </ThemeContext.Provider>
  );
}
export default App;
```

How It Works

Step 1
Create context
Step 2
Wrap provider
Step 3
Access using useContext
Real Project Usage
Context API used in:

- login authentication
- cart system
- user profile
- theme switcher
- dashboards
- employee portal
- LMS systems

This is very useful for your employee management or LMS projects where login user details need to be accessed in many pages.
Tasks (Detailed)
Task 1
Create simple context
Task 2
Pass username globally
Task 3
Display username in child component
Task 4
Create theme context
Task 5
Show dark/light mode text
Task 6
Create student context
Task 7
Create employee context
Task 8
Create cart context
Task 9
Use multiple child components
Task 10

Build global dashboard context

Mini Projects (Detailed)

Mini Project 1: User Login Context

Share:

- username
- role
- email

Across components.

Mini Project 2: Theme Switcher

Global theme:

- dark
- light

Using context.

Mini Project 3: Cart Management

Share cart items between:

- products page
- cart page

Mini Project 4: LMS User Context

Share:

- student name
- enrolled courses
- profile info

Day 40

Forms Handling

What is Forms Handling in React?

In React, forms handling means managing user input fields such as:

- text boxes
- email inputs
- password inputs
- dropdowns
- radio buttons
- checkboxes
- submit buttons

React handles forms using **state**.

Simple Meaning

Forms Handling = Taking user input and processing it in React

Why Forms are Important?

Almost every application needs forms.

Examples:

- login form
- registration form
- employee form
- leave request form
- job application
- course enrollment
- contact form

Since you're building systems like LMS, job portal, and employee management, form handling is one of the most practical React topics.

Basic Form Example

A simple form may have:

- name
- email
- password
- submit

React captures these values and stores them in state.

Controlled Components

React forms usually use:

Controlled Components

That means input values are controlled by React state.

Basic Syntax

```
import { useState } from "react";
```

Single Input Example

```
import { useState } from "react";

function App() {
  const [name, setName] = useState("");

  return (
    <div>
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
```

```
        />

        <h2>{name}</h2>
      </div>
  );
}
```

How It Works

- user types input
- onChange event triggers
- state updates
- UI displays new value

Multiple Inputs

```
import { useState } from "react";

function App() {
  const [form, setForm] = useState({
    name: "",
    email: ""
  });

  function handleChange(e) {
    setForm({
      ...form,
      [e.target.name]: e.target.value
    });
  }

  return (
    <div>
      <input name="name" onChange={handleChange} />
      <input name="email" onChange={handleChange} />
    </div>
  );
}
```

Form Submit

```
function handleSubmit(e) {
  e.preventDefault();
```

```
  console.log(form);
}
```

Full Example

```
<form onSubmit={handleSubmit}>
  <button type="submit">Submit</button>
</form>
```

Why preventDefault()?

It stops page reload.

Validation Example

```
if (name === "") {
  alert("Enter name");
}
```

Common Validation

- empty field
- email format
- password length
- required fields

Real Project Usage

Forms are used in:

- login systems
- registration pages
- employee forms
- job applications
- course enrollment
- contact forms
- feedback systems

Very useful for the Django-based apps you're already building.

Full Example

```
import { useState } from "react";

function App() {
  const [name, setName] = useState("");

  function handleSubmit(e) {
    e.preventDefault();
    alert(name);
  }
```

```
 return (
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />
<button type="submit">Submit</button>
      </form>
   );
}
export default App;
```

Tasks (Detailed)

Task 1

Create name input form

Task 2

Create email input

Task 3

Create password input

Task 4

Display input values

Task 5

Handle submit event

Task 6

Add validation

Task 7

Create registration form

Task 8

Create login form

Task 9

Create employee form

Task 10

Create full dynamic form

Mini Projects (Detailed)

Mini Project 1: Student Registration Form

Fields:

- name

- age
- course

Mini Project 2: Login Form

Fields:

- email
- password

Mini Project 3: Employee Form

Fields:

- employee name
- department
- role

Mini Project 4: Course Enrollment Form

Fields:

- student name
- course
- duration

Day 41

React Mini projecs

Mini Projects

Mini Project 1: Student Manager

Features:

- add student
- delete
- search

Mini Project 2: Product Manager

Features:

- add product
- delete
- filter

Mini Project 3: Course Manager

Features:

- add course
- remove
- search

Mini Project 4: Task Manager

Features:

- add task
- delete
- mark complete

Day 42

Revision

Topics to Revise

React basics covered so far:

- React introduction
- Components
- JSX
- Props
- State
- Conditional rendering
- Lists
- Events
- API calls
- Routing
- Context API
- Forms handling
- Mini project practice

All of these are core concepts in React development.

Why Revision is Important?

Revision helps:

remember syntax
improve coding speed
understand concepts deeply
prepare for mini projects
build confidence

Without revision:

- topics may be forgotten quickly

Theory Revision

React

React is a JavaScript library used for building dynamic user interfaces.

Components

Reusable UI blocks.

Examples:

- Navbar
- Footer
- Cards
- Forms

JSX

Allows writing HTML inside JavaScript.

Props

Used to pass data between components.

State

Stores dynamic data inside component.

Conditional Rendering

Displays UI based on conditions.

Lists

Used to display arrays using map().

Events

User actions:

- click
- input
- submit

API Calls

Fetch server data using fetch() + useEffect().

Routing

Navigate between pages using React Router.

Context API

Share global data between components.

Forms

Handle user input using state.